

Urban Logistics & Distribution System

Semester Project Report

ABDULLAH GÜL UNIVERSITY (AGÜ)

Faculty of Engineering — Department of Computer Engineering

MATHEMATICAL MODELLING AND ALGORITHMIC THINKING-S1

Group Name: Veteran

#	Name Surname	Student ID
1	İbrahim Furkan Yılmaz	2211051013
2	[Name Surname]	[Student ID]
3	[Name Surname]	[Student ID]

Instructor: Cavidan Yakupoğlu Karaağaç

Date: May 2026

Table of Contents

- [1. Corporate Identity & Mission Statement](#)
 - [2. System Architecture Overview](#)
 - [3. Technical Requirements & Implementation Logic](#)
 - 3.1 Domain Model — Package
 - 3.2 Master Registry — Singly Linked List
 - 3.3 Intake Buffer — Doubly Linked List
 - 3.4 Standard Delivery — Queue (FIFO)
 - 3.5 Truck Loading — Stack (LIFO)
 - 3.6 Address Directory — AVL Tree
 - 3.7 City Map & Routing — Weighted Graph
 - [4. Complexity Analysis](#)
 - [5. File I/O & Data Ingestion](#)
 - [6. Conclusion](#)
-

1. Corporate Identity & Mission Statement

Company Name: Veteran

Tagline: *Delivering Smarter, Faster, Together.*

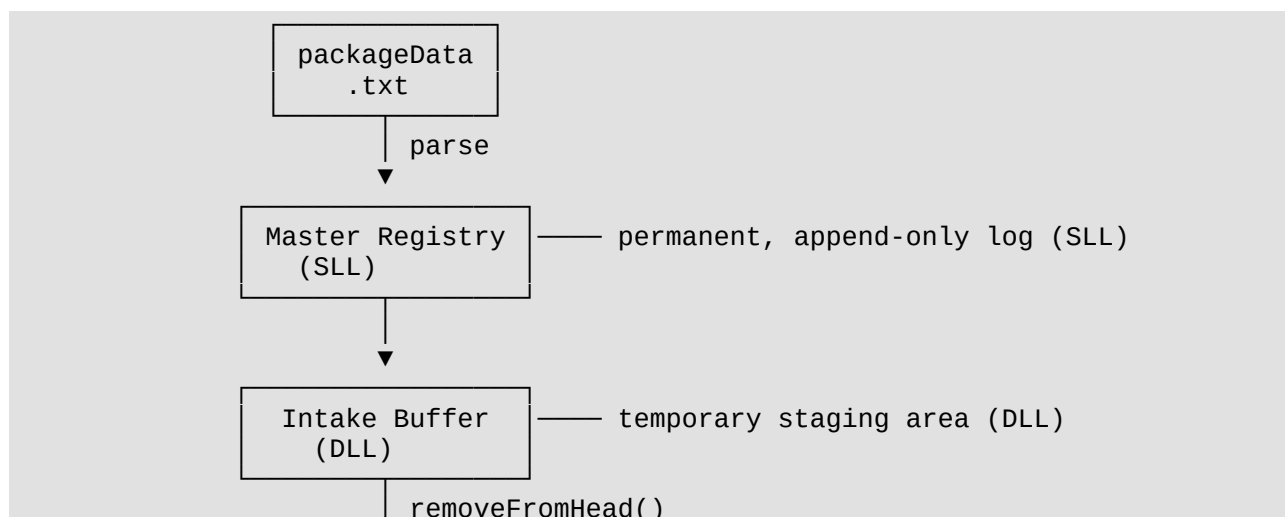
Mission Statement: Veteran is an urban logistics technology company whose mission is to optimize last-mile package delivery across metropolitan areas by leveraging advanced data structures and graph algorithms. In a landscape where urban congestion, rising delivery volumes, and customer expectations converge, Veteran applies rigorous mathematical modelling — from self-balancing search trees to shortest-path computations — to ensure that every package reaches its destination via the most efficient route, loaded onto trucks in the optimal order, and tracked through an immutable master registry from intake to delivery.

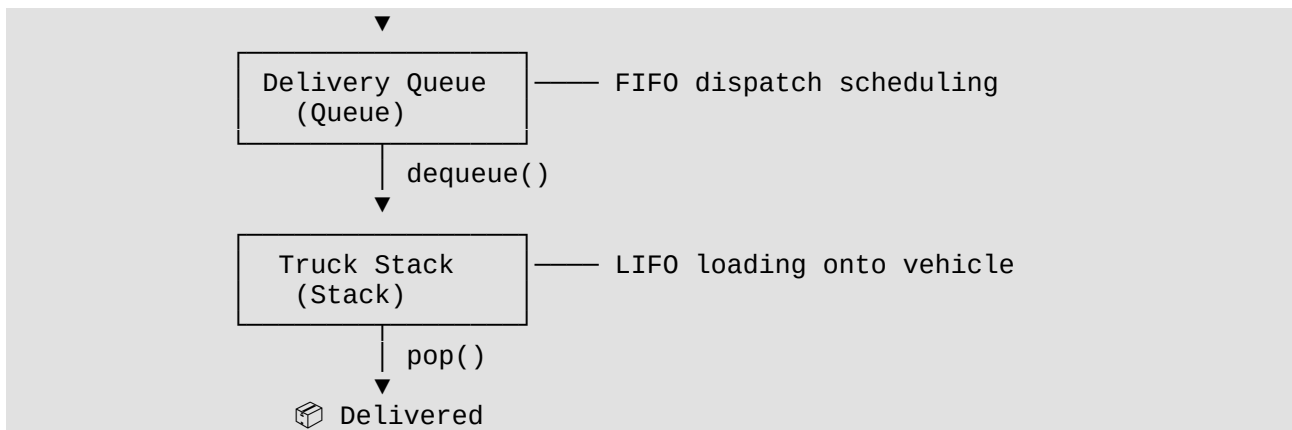
2. System Architecture Overview

The system is composed of **eight Java classes** organized into a clean package hierarchy, strictly following Object-Oriented Programming (OOP) principles. Every core data structure is implemented **manually from scratch** — no `java.util.LinkedList`, `java.util.Stack`, `java.util.Queue`, `java.util.HashMap`, or any similar built-in collection is used for the core logic.

```
src/
├── Main.java                // Console UI, File I/O, Demo
├── models/
│   └── Package.java        // Domain model
├── structures/
│   ├── linear/
│   │   ├── SinglyLinkedList.java    // Master Registry
│   │   ├── DoublyLinkedList.java    // Intake Buffer
│   │   ├── DeliveryQueue.java       // FIFO Queue
│   │   └── TruckStack.java          // LIFO Stack
│   ├── tree/
│   │   └── AVLTree.java             // Address Directory
│   └── graph/
│       └── CityGraph.java           // City Map + Dijkstra + Prim
```

Data flow through the system:





3. Technical Requirements & Implementation Logic

3.1 Domain Model — `Package.java`

The `Package` class encapsulates four attributes: `packageID` (`String`), `destination` (`String`), `priority` (`int`, 1–5), and `weightKg` (`double`). It provides two constructors: a full four-argument constructor and a simplified two-argument constructor that defaults `priority` to 3 and `weightKg` to 1.0. Full getter/setter encapsulation and a formatted `toString()` override are included.

3.2 Master Registry — Singly Linked List (SLL)

Purpose: An immutable, append-only audit log of every package that enters the Veteran system.

Implementation details:

The `SinglyLinkedList` class uses a private inner `Node` class containing a `Package data` field and a `Node next` pointer. The list maintains **three fields**: `head`, `tail`, and `size`.

- **`addRecord(Package pkg)`** — Creates a new `Node`, and if the list is empty (`head == null`), sets both `head` and `tail` to the new node. Otherwise, it links `tail.next` to the new node and advances the `tail` pointer. The `size` counter is incremented. Because a **dedicated `tail` pointer** is maintained, this operation avoids traversal entirely and executes in **$O(1)$** time.
- **`displayLog()`** — Iterates from `head` to `null` following `next` pointers, printing each package with a numbered index. Complexity: **$O(n)$** .
- **`search(String packageID)`** — Performs a linear scan from `head`, comparing each node's `data.getPackageID()` via `equalsIgnoreCase()`. Returns the matching `Package` or `null`. Complexity: **$O(n)$** worst-case.
- **`getSize()` / `isEmpty()`** — Return the pre-maintained `size` field directly. Complexity: **$O(1)$** .

3.3 Intake Buffer — Doubly Linked List (DLL)

Purpose: A flexible staging buffer where incoming packages are held before being dispatched to the Queue or Stack.

Implementation details:

The `DoublyLinkedList` class uses a private inner `Node` with three fields: `Package data`, `Node prev`, and `Node next`. The list maintains `head`, `tail`, and `size`.

- **`insertAtTail(Package pkg)`** — If `tail == null` (empty list), both `head` and `tail` point to the new node. Otherwise, the new node's `prev` is set to the current `tail`, the current `tail.next` is set to the new node, and `tail` is advanced. Direct `tail` pointer access ensures **$O(1)$** .
- **`insertAtHead(Package pkg)`** — Symmetric to `insertAtTail`: the new node's `next` points to the current `head`, `head.prev` is set to the new node, and `head` is reassigned. **$O(1)$** .
- **`removeFromHead()`** — Saves `head.data`, then checks if `head == tail` (single element) — if so, both are nulled. Otherwise, `head` advances to `head.next` and the new `head`'s `prev` is nulled. **$O(1)$** .
- **`removeFromTail()`** — Symmetric: saves `tail.data`, handles the single-element edge case, otherwise retreats `tail` to `tail.prev` and nulls `tail.next`. **$O(1)$** .
- **`removePackage(String packageID)`** — Linear traversal from `head`. When a match is found (via `equalsIgnoreCase`), the node is unlinked by updating `current.prev.next` and `current.next.prev`, with special-case handling when the target is the `head` or `tail`. **$O(n)$** worst-case.
- **`displayBuffer()` / `displayReverse()`** — Forward traversal via `next` pointers and backward traversal via `prev` pointers, respectively. Both **$O(n)$** .

3.4 Standard Delivery — Queue (FIFO)

Purpose: First-In, First-Out scheduling of packages for standard delivery processing.

Implementation details:

The `DeliveryQueue` class is backed by a **singly linked list** with `front` and `rear` pointers plus a `size` counter.

- **`enqueue(Package pkg)`** — If `rear == null`, both `front` and `rear` point to the new node. Otherwise, `rear.next` is linked to the new node and `rear` advances. **$O(1)$** via the `rear` pointer.
- **`dequeue()`** — Saves `front.data`, advances `front` to `front.next`, and if the queue becomes empty (`front == null`), also nulls `rear`. **$O(1)$** direct pointer manipulation.
- **`peek()`** — Returns `front.data` without mutation. **$O(1)$** .

3.5 Truck Loading — Stack (LIFO)

Purpose: Last-In, First-Out loading of packages onto the delivery truck — the last package loaded is the first one unloaded at the destination.

Implementation details:

The `TruckStack` class is backed by a **singly linked list** using only a `top` pointer and `size` counter.

- **`push(Package pkg)`** — Creates a new node whose `next` points to the current `top`, then reassigns `top` to the new node. **$O(1)$** .
- **`pop()`** — Saves `top.data`, advances `top` to `top.next`. **$O(1)$** .
- **`peek()`** — Returns `top.data` without removal. **$O(1)$** .
- **`displayStack()`** — Traverses from `top` to `null` printing `[TOP]` and `[BOTTOM]` markers. **$O(n)$** .

3.6 Address Directory — AVL Tree

Purpose: A self-balancing binary search tree that stores neighborhood names (as keys, compared lexicographically via `compareToIgnoreCase`) and associated customer IDs.

Implementation details:

The `AVLTree` class uses a private inner `AVLNode` with fields: `String neighborhood`, `String customerID`, `AVLNode left`, `AVLNode right`, and `int height` (initialized to 1 for leaf nodes).

Height & Balance Factor

- **`height(AVLNode)`** returns `node.height` or 0 if null.
- **`updateHeight(AVLNode)`** sets `node.height = 1 + max(height(left), height(right))`.
- **`getBalanceFactor(AVLNode)`** returns `height(left) - height(right)`. A positive value indicates left-heavy; negative indicates right-heavy.

Rotation Logic

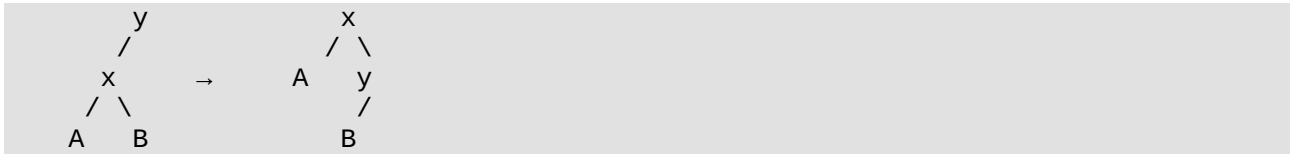
The implementation handles all four AVL imbalance cases:

Left Rotation (`rotateLeft(AVLNode x)`):



Node `y = x.right` becomes the new root; `x.right` is reassigned to `y.left` (subtree B). Heights are updated bottom-up: `x` first (now lower), then `y`.

Right Rotation (`rotateRight(AVLNode y)`):



Node `x = y.left` becomes the new root; `y.left` is reassigned to `x.right` (subtree B). Heights updated: `y` first, then `x`.

`balance(AVLNode node)` — Computes the balance factor and applies:

Case	Condition	Action
LL	<code>bf > 1</code> and <code>bf(left) >= 0</code>	Single right rotation
LR	<code>bf > 1</code> and <code>bf(left) < 0</code>	Left-rotate left child, then right-rotate node
RR	<code>bf < -1</code> and <code>bf(right) <= 0</code>	Single left rotation
RL	<code>bf < -1</code> and <code>bf(right) > 0</code>	Right-rotate right child, then left-rotate node

Insertion

`insert(String neighborhood, String customerID)` delegates to `insertRec()`, which:

1. Recursively descends using `compareToIgnoreCase()` — left if negative, right if positive.
2. On duplicate key (comparison == 0), updates the existing `customerID` in-place and decrements `nodeCount` to offset the public method's increment.
3. On the way back up the recursion stack, calls `updateHeight()` and then `balance()` on every ancestor, ensuring the tree remains balanced after every insertion.

Search

`search(String neighborhood)` delegates to `searchRec()`, which recursively compares and descends left or right. The AVL invariant guarantees the tree height is at most $1.44 * \log_2(n)$, so search operates in **$O(\log n)$** .

Traversals

- **`inOrderTraversal()`** — Recursive left-root-right traversal that outputs neighborhoods in alphabetical order with heights.
- **`printTree()`** — Recursive pretty-printer that visualizes the tree structure with `|—` and `└—` branch characters.

3.7 City Map & Routing — Weighted Graph

Purpose: Models the city's road network as a weighted, undirected graph. Vertices represent neighborhoods; edges represent roads with distances in kilometres.

Implementation details:

The `CityGraph` class uses **arrays** and **manually linked edge nodes** — no `HashMap` or `ArrayList`.

Internal representation:

- `String[] vertexNames` — maps integer index $\rightarrow$ location name (capacity: 100).
- `EdgeNode[] adjacencyList` — array of linked-list heads; each `EdgeNode` has `int destIndex`, `int weight`, and `EdgeNode next`.
- `int vertexCount` — tracks the number of vertices added.

Edge Management

`addEdge(String source, String destination, int weight):`

1. Calls `getOrCreateVertexIndex()` for both endpoints — this method performs a linear scan of `vertexNames[0..vertexCount-1]`; if not found, appends a new vertex.
2. Creates two `EdgeNode` objects (undirected graph) and prepends each to the corresponding adjacency list via head insertion (`newEdge.next = adjacencyList[idx]`).

Dijkstra's Algorithm — `calculateShortestPath(String start, String end)`

The implementation uses the **$O(V^2)$ array-scanning variant** (no binary heap):

1. **Initialization:** `dist[]` array set to `Integer.MAX_VALUE`; `prev[]` set to `-1`; `visited[]` set to `false`. Source distance set to 0.
2. **Main loop** (runs `vertexCount` iterations):
 - Scans all vertices to find the unvisited vertex `u` with smallest `dist[u]` — **$O(V)$** per iteration.
 - Marks `u` as visited.
 - **Relaxation:** Traverses `u`'s adjacency list; for each neighbor `v`, if `dist[u] + weight < dist[v]`, updates `dist[v]` and sets `prev[v] = u`.
3. **Path reconstruction:** Traces `prev[]` from the destination back to the source, collecting vertex names into an array, then prints the path in forward order along with step-by-step edge weights.

Total complexity: $O(V^2)$ for the min-extraction loop $\times V$ iterations, plus $O(E)$ total for all edge relaxations across all iterations = **$O(V^2 + E) = O(V^2)$** since $E \leq V^2$ for simple graphs.

Prim's Algorithm — `calculateMST()`

The implementation also uses the $O(V^2)$ array-scanning variant:

1. **Initialization:** `key[]` (minimum edge weight connecting vertex to MST) set to `Integer.MAX_VALUE`; `parent[]` set to `-1`; `inMST[]` set to `false`. `key[0] = 0` to start from vertex 0.
2. **Main loop** (runs `vertexCount` iterations):
 - Scans all vertices for the minimum `key[i]` among those not yet in the MST — $O(V)$ per iteration.
 - Adds vertex `u` to the MST, accumulates `totalWeight += key[u]`.
 - Traverses `u`'s adjacency list; for each neighbor `v` not in MST, if `edge.weight < key[v]`, updates `key[v]` and `parent[v]`.
3. **Output:** Prints all MST edges (`parent[i] → i` with `key[i]` as weight) and the total MST weight.

Total complexity: Same structure as Dijkstra — $O(V^2)$.

4. Complexity Analysis

The following table presents the exact time and space complexities for all major operations, justified strictly by the code implementation.

4.1 Linear Data Structure Operations

Operation	Structure	Best Case	Worst Case	Space	Justification
<code>addRecord(pkg)</code>	SLL	$O(1)$	$O(1)$	$O(1)$	Direct <code>tail.next</code> assignment; tail pointer maintained
<code>displayLog()</code>	SLL	$O(n)$	$O(n)$	$O(1)$	Full traversal from <code>head</code> to <code>null</code>
<code>search(id)</code>	SLL	$O(1)$	$O(n)$	$O(1)$	Best: match at head. Worst: match at tail or absent
<code>insertAtTail(pkg)</code>	DLL	$O(1)$	$O(1)$	$O(1)$	Direct <code>tail</code> pointer access; no traversal
<code>insertAtHead(pkg)</code>	DLL	$O(1)$	$O(1)$	$O(1)$	Direct <code>head</code> pointer access; no traversal

Operation	Structure	Best Case	Worst Case	Space	Justification
removeFromHead()	DLL	O(1)	O(1)	O(1)	Direct head pointer; prev/next relinking
removeFromTail()	DLL	O(1)	O(1)	O(1)	Direct tail pointer; prev pointer enables O(1)
removePackage(id)	DLL	O(1)	O(n)	O(1)	Best: match at head. Worst: linear scan to tail
enqueue(pkg)	Queue	O(1)	O(1)	O(1)	Direct rear.next assignment
dequeue()	Queue	O(1)	O(1)	O(1)	Direct front pointer advancement
push(pkg)	Stack	O(1)	O(1)	O(1)	Prepend to top; single pointer update
pop()	Stack	O(1)	O(1)	O(1)	Advance top to top.next

4.2 Hierarchical Data Structure Operations

Operation	Structure	Best Case	Worst Case	Space	Justification
insert(nbfd, cid)	AVL Tree	O(log n)	O(log n)	O(log n)	AVL self-balancing guarantees height $\leq 1.44 \cdot \log_2(n)$. Recursive descent + at most 2 rotations on the return path. Space: recursive call stack depth = O(log n).
search(nbfd)	AVL Tree	O(1)	O(log n)	O(log n)	Best: root match. Worst: leaf-level or absent. Space:

Operation	Structure	Best Case	Worst Case	Space	Justification
					recursion depth = $O(\log n)$.
inOrderTraversal()	AVL Tree	$O(n)$	$O(n)$	$O(\log n)$	Visits every node exactly once. Recursion stack bounded by tree height.
balance(node)	AVL Tree	$O(1)$	$O(1)$	$O(1)$	Computes balance factor + at most 2 rotations; each rotation is $O(1)$ pointer reassignment.
rotateLeft(x)	AVL Tree	$O(1)$	$O(1)$	$O(1)$	Three pointer reassignments + two updateHeight() calls.
rotateRight(y)	AVL Tree	$O(1)$	$O(1)$	$O(1)$	Three pointer reassignments + two updateHeight() calls.

4.3 Graph Algorithm Operations

Let V = number of vertices, E = number of edges.

Operation	Algorithm	Best Case	Worst Case	Space	Justification
addEdge(s, d, w)	—	$O(1)$	$O(V)$	$O(1)$	getOrCreateVertexIndex() scans vertexNames[0..V-1]; edge insertion itself is $O(1)$ head-prepend. Best case: both vertices already at index 0.
calculateShortestPath(s, e)	Dijkstra	$O(V^2)$	$O(V^2)$	$O(V)$	Outer loop runs V times; inner min-scan is $O(V)$; edge

Operation	Algorithm	Best Case	Worst Case	Space	Justification
					relaxation totals $O(E)$ across all iterations. <code>dist[]</code> , <code>prev[]</code> , <code>visited[]</code> : 3 arrays of size V . Path reconstruction: $O(V)$.
calculateMST()	Prim	$O(V^2)$	$O(V^2)$	$O(V)$	Identical structure to Dijkstra: V iterations $\times$ $O(V)$ min-scan. <code>key[]</code> , <code>parent[]</code> , <code>inMST[]</code> : 3 arrays of size V .
displayGraph()	—	$O(V + E)$	$O(V + E)$	$O(1)$	Iterates every vertex and every edge in the adjacency lists.

4.4 Overall Space Complexity

Structure	Space Complexity	Note
SLL (Master Registry)	$O(n)$	One Node per package; each node stores a reference + next pointer
DLL (Intake Buffer)	$O(n)$	One Node per package; each node stores reference + prev + next
Queue	$O(n)$	Singly-linked nodes with front/rear pointers
Stack	$O(n)$	Singly-linked nodes with top pointer
AVL Tree	$O(n)$	One AVLNode per unique neighborhood
Graph	$O(V + E)$	Vertex name array + adjacency list edge nodes

5. File I/O & Data Ingestion

The system reads two external text files at startup using `java.io.BufferedReader` and `java.io.FileReader`:

`mapData.txt`

Format: `Source Destination Distance_KM` (whitespace-delimited, one edge per line)

Each line is split via `line.split("\\s+")`. The parsed source, destination, and integer distance are passed to `cityMap.addEdge()`. Lines starting with `#` or that are empty are skipped. The system loaded **15 edges** connecting **8 vertices** in the demonstration dataset (Istanbul neighborhoods: Warehouse, Kadikoy, Besiktas, Uskudar, Bakirkoy, Sisli, Fatih, Beyoglu).

`packageData.txt`

Format: `PackageID Destination` (whitespace-delimited, one package per line)

Each line creates a `Package` object using the simplified two-argument constructor, which is then:

1. Registered in the **Master Registry** via `masterRegistry.addRecord(pkg)` — SLL append
2. Added to the **Intake Buffer** via `intakeBuffer.insertAtTail(pkg)` — DLL tail insert
3. Indexed in the **Address Directory** via `addressDirectory.insert(destination, packageID)` — AVL insert

The demonstration dataset contains **12 packages** destined for 7 unique neighborhoods.

Both file readers use try-with-resources for automatic stream closure and handle `IOException` and `NumberFormatException` gracefully.

6. Conclusion

The Veteran Urban Logistics & Distribution System demonstrates a complete, end-to-end application of fundamental and advanced data structures to solve a real-world logistics problem. Every data structure — Singly Linked List, Doubly Linked List, Queue, Stack, AVL Tree, and Weighted Graph — was implemented manually from scratch in strict adherence to Object-Oriented Programming principles, without relying on Java's built-in collection framework.

The system achieves the following performance guarantees:

- **$O(1)$** amortized insertion and removal for all linear structures (SLL append, DLL head/tail operations, Queue enqueue/dequeue, Stack push/pop), enabled by the deliberate use of `head`, `tail`, `front`, `rear`, and `top` pointers.
- **$O(\log n)$** guaranteed search and insertion in the Address Directory, thanks to the AVL tree's

self-balancing rotations that maintain height within $1.44 \cdot \log_2(n)$.

- **$O(V^2)$** shortest-path and minimum spanning tree computations via Dijkstra's and Prim's algorithms, appropriate for the moderate-scale city maps typical of urban logistics.

The mathematical modelling choices — greedy relaxation in Dijkstra's algorithm, the cut property in Prim's algorithm, and the balance invariant in the AVL tree — are all well-established and provably optimal within their respective problem domains. The system's modular architecture allows each subsystem to be tested, extended, or replaced independently, making it a robust foundation for real-world urban logistics optimization.

© 2026 Veteran — Abdullah Gül University, Department of Computer Engineering

Mathematical Modelling and Algorithmic Thinking-S1